

1. Fetch (取指) 部件信号生成逻辑

Fetch部件的核心任务是从指令存储器中读取指令字节，并根据第一个字节（指令码`icode`和功能码`ifun`）确定指令的后续部分，最终计算出下一条指令的地址`valP`。

灰色框中的信号是根据PC值从指令存储器中读取并解析得到的，其生成逻辑如下：

- **icode:ifun:**
 - 逻辑: `icode:ifun = M_1[PC]`
 - 从程序计数器PC指向的内存地址读取1个字节。该字节的高4位是指令码`icode`，低4位是功能码`ifun`。`icode`决定了指令的类型，`ifun`则用于区分同一指令类型的不同操作（如`OPq`指令的具体算术逻辑运算）。
- **rA:rB:**
 - 逻辑: `rA:rB = M_1[PC+1]` (当 `icode` in {`RRMOVQ`, `CMOVXX`, `OPQ`, `PUSHQ`, `POPQ`, `IRMOVQ`, `RMMOVQ`, `MRMOVQ`} 时)
 - 如果指令包含寄存器操作数（由`icode`判断），则从PC+1的地址读取1个字节。该字节的高4位是寄存器A的编号`rA`，低4位是寄存器B的编号`rB`。如果指令不使用寄存器，这两个字段可能为`0xF` (`RNONE`)。
- **valC:**
 - 逻辑: `valC = M_8[PC+1]` (当 `icode` in {`IRMOVQ`, `JXX`, `CALL`} 时) 或 `valC = M_8[PC+2]` (当 `icode` in {`RMMOVQ`, `MRMOVQ`} 时)
 - 如果指令包含一个8字节的立即数`valC`（如`irmovq`的立即数，`jxx/call`的目标地址，或`rmmovq/mrmovq`的偏移量），则从指令的相应位置（PC+1或PC+2）读取这8个字节。
- **valP:**
 - 逻辑: `valP = PC + instr_len`
 - `valP`是下一条指令的地址。它的值等于当前PC加上当前指令的长度`instr_len`。指令长度由`icode`决定：
 - `halt, nop, ret`: 1字节
 - `rrmovq, cmovXX, opq, pushq, popq`: 2字节
 - `irmovq, jXX, call`: 9字节
 - `rmmovq, mrmovq`: 10字节

2. Decode (译码) 部件信号用途

Decode部件接收来自Fetch部件的信号，并从寄存器文件（Register File）中读取操作数。它的主要任务是为执行阶段（Execute）准备数据源。

- **srcA:**
 - 用途: 指定寄存器文件“A”端口的读地址。它决定了哪个寄存器的值将被读出并作为`valA`。
 - 来源: 通常是指令中的`rA`字段，但在某些情况下（如`pushq`和`ret`）会是栈指针`RSP`。
- **srcB:**

- 用途: 指定寄存器文件“B”端口的读地址。它决定了哪个寄存器的值将被读出并作为`valB`。
- 来源: 通常是指令中的`rB`字段。
- **dstE:**
 - 用途: 指定寄存器文件“E”端口的写地址。ALU的计算结果`valE`将写入`dstE`所指定的寄存器。
 - 来源: 通常是指令中的`rB`字段, 但在`call/ret`等指令中会是栈指针`RSP`。
- **dstM:**
 - 用途: 指定寄存器文件“M”端口的写地址。从内存中读出的数据`valM`将写入`dstM`所指定的寄存器。
 - 来源: 通常是指令中的`rA`字段。
- **valA:**
 - 用途: 它是ALU的两个操作数之一。
 - 从寄存器文件`srcA`端口读出的值。
- **valB:**
 - 用途: 它也是ALU的两个操作数之一。
 - 从寄存器文件`srcB`端口读出的值。

3. `srcA` 和 `srcB` 生成逻辑

`srcA`和`srcB`决定了从寄存器文件中读取哪些寄存器的值。

`srcA` 生成逻辑分析

`srcA`的选择逻辑如下:

```
word srcA = [
    icode in { RRMovQ, RMMovQ, OPQ, PUSHQ } : rA;
    icode in { POPQ, RET } : RSP;
    1 : RNONE; // 其他指令不需要从A端口读
];
```

- **icode in { RRMovQ, RMMovQ, OPQ, PUSHQ } : rA:** 对于这些指令, `valA`是源操作数。例如, `rrmovq rA, rB`中`rA`是源, `pushq rA`中`rA`是要压栈的源。因此, `srcA`被设置为指令中的`rA`字段。
- **icode in { POPQ, RET } : RSP:** `popq`和`ret`指令都需要读取栈顶内容。在执行前, 它们需要先读取当前的栈指针`RSP`来计算访存地址。因此, `srcA`被设置为`RSP`。
- **1 : RNONE:** 对于其他指令 (如`nop`, `halt`, `irmovq`, `call`等), 它们不需要从寄存器文件A端口读取数据, 因此`srcA`设为`RNONE` (值为`0xF`), 表示不读取。

`srcB` 生成逻辑分析

`srcB`的选择逻辑如下:

```
word srcB = [
    icode in { OPQ, RMMOVQ, MRMOVQ } : rB;
    icode in { PUSHQ, POPQ, CALL, RET } : RSP;
    1 : RNONE; // 其他指令不需要从B端口读
];
```

- **icode in { OPQ, RMMOVQ, MRMOVQ } : rB:**
 - **OPq rA, rB:** rB既是源操作数也是目的寄存器。valB作为ALU的一个输入。
 - **rmmovq rA, D(rB)** 和 **mrmmovq D(rB), rA:** rB作为基址寄存器，其值valB用于计算内存地址。
 - 因此，srcB被设置为指令中的rB字段。
- **icode in { PUSHQ, POPQ, CALL, RET } : RSP:** 这四条指令都涉及对栈的操作，需要使用栈指针RSP作为基址来计算内存地址。
 - **pushq:** RSP减小后作为写内存的地址。
 - **popq:** RSP作为读内存的地址，之后RSP增大。
 - **call:** RSP减小，用于保存返回地址。
 - **ret:** RSP作为读内存地址，以获取返回地址。
 - 因此，srcB被设置为RSP。
- **1 : RNONE:** 其他指令（如nop, halt, rrmovq, irmovq）不需要从B端口读取数据，srcB设为RNONE。

4. dstE 和 dstM 生成逻辑

dstE和dstM决定了计算结果或内存读取结果应写入哪个目标寄存器。

dstE 生成逻辑

dstE (Destination for valE) 用于指定ALU计算结果valE的写入目标寄存器。

```
word dstE = [
    icode in { RRMOVQ, IRMOVQ, OPQ } : rB;
    icode in { PUSHQ, POPQ, CALL, RET } : RSP;
    1 : RNONE; // 其他指令不写回valE
];
```

- **icode in { RRMOVQ, IRMOVQ, OPQ } : rB:**
 - **rrmovq rA, rB:** valA（从rA读出）通过ALU（通常是A+0操作）后得到valE，需要写入rB。
 - **irmovq V, rB:** 立即数valC通过ALU（V+0）后得到valE，需要写入rB。
 - **OPq rA, rB:** valA和valB的运算结果valE需要写回rB。
 - 因此，dstE被设置为指令中的rB字段。
- **icode in { PUSHQ, POPQ, CALL, RET } : RSP:** 这四条指令都会修改栈指针RSP。ALU负责计算新的RSP值（加8或减8），结果为valE，需要写回RSP寄存器。因此，dstE被设置为RSP。
- **1 : RNONE:** 其他指令（如halt, nop, rmmovq, mrmmovq, jxx）不通过valE更新寄存器，dstE设为RNONE。

dstM 生成逻辑分析

dstM (Destination for valM) 用于指定从内存中读取的数据 **valM** 的写入目标寄存器。

```
word dstM = [
    icode in { MRM0VQ, POPQ } : rA;
    1 : RNONE; // 其他指令不写回valM
];
```

- **icode in { MRM0VQ, POPQ } : rA:**
 - **mrmovq D(rB), rA:** 从内存地址 **D+valB** 读取数据 **valM**，并将其写入 **rA** 寄存器。
 - **popq rA:** 从栈顶（地址为 **valA**，即旧的 **RSP**）读取数据 **valM**，并将其写入 **rA** 寄存器。
 - 因此，对于这两条指令，**dstM** 被设置为指令中的 **rA** 字段。
- **1 : RNONE:** 只有 **mrmovq** 和 **popq** 需要将内存数据写回寄存器。其他所有指令都不执行此操作，因此 **dstM** 设为 **RNONE**。

5. ALU A 和 ALU B 生成逻辑

ALU A 和 **ALU B** 是送入 ALU（算术逻辑单元）的两个操作数。

ALU A 生成逻辑

ALU A 的选择逻辑如下：

```
word aluA = [
    icode in { RRM0VQ, OPQ } : valA;
    icode in { IRMOVQ, RMM0VQ, MRM0VQ } : valC;
    icode in { CALL, PUSHQ } : -8;
    icode in { RET, POPQ } : 8;
    1 : 0; // 其他指令该输入未使用
];
```

- **icode in { RRM0VQ, OPQ } : valA:**
 - **rrmovq rA, rB:** 需要将 **valA**（从 **rA** 读出）传递给 **valE**，ALU 执行 **valA + 0**。
 - **OPq rA, rB:** **valA** 是 ALU 运算的一个源操作数。
- **icode in { IRMOVQ, RMM0VQ, MRM0VQ } : valC:**
 - **irmovq V, rB:** 需要将立即数 **valC** 传递给 **valE**，ALU 执行 **valC + 0**。
 - **rmmovq rA, D(rB)** 和 **mrmovq D(rB), rA:** 需要计算内存地址 **D + valB**，其中 **D** 是立即数 **valC**。
- **icode in { CALL, PUSHQ } : -8:** **call** 和 **pushq** 指令需要将栈指针 **RSP** 减 8。ALU 通过计算 **valB + (-8)** 来实现。
- **icode in { RET, POPQ } : 8:** **ret** 和 **popq** 指令需要将栈指针 **RSP** 加 8。ALU 通过计算 **valA + 8** 来实现。
- **1 : 0:** 对于其他指令，此输入不重要，可以设为 0。

ALU B 生成逻辑分析

ALU B 的选择逻辑如下：

```
word aluB = [
  icode in { RMMOVQ, MRMOVQ, OPQ, CALL, PUSHQ, RET, POPQ } : valB;
  icode in { RRMOVQ, IRMOVQ } : 0;
  1 : 0; // 其他指令该输入未使用
];
```

- **icode in { RMMOVQ, MRMOVQ, OPQ, CALL, PUSHQ, RET, POPQ } : valB:**
 - **rmmovq, mrmovq:** valB (从rB读出) 是基址, 用于计算内存地址 $\text{valC} + \text{valB}$ 。
 - **OPq:** valB (从rB读出) 是ALU运算的另一个源操作数。
 - **call, pushq, ret, popq:** valB (从RSP读出) 是当前的栈指针, 用于计算新的栈指针值。
- **icode in { RRMOVQ, IRMOVQ } : 0:**
 - **rrmovq:** ALU执行 $\text{valA} + 0$, 将valA传递到valE。
 - **irmovq:** ALU执行 $\text{valC} + 0$, 将valC传递到valE。
 - 在这两种情况下, ALU B输入都应设为0。
- **1 : 0:** 对于其他指令 (如 **halt, nop, jxx**) , 此输入不影响结果, 可以设为0。

6. Memory (内存) 部件信号用途

Memory部件负责读或写数据存储器。

- **Mem. Addr (内存地址):**
 - 用途: 指定要进行读或写操作的内存地址。
 - 来源: 通常是ALU的计算结果valE (例如, **rmmovq**或**mrmovq**计算出的地址) , 或者是valA (例如, **popq**或**ret**指令中的旧栈顶地址) 。
- **Mem. Data In (写入数据):**
 - 用途: 提供要写入内存的数据。
 - 来源: valA。例如, 在**rmmovq rA, D(rB)**指令中, valA (从rA读出) 是要写入内存的数据。在**call**指令中, valP (返回地址) 通过valA通路提供给内存写入。
- **Mem. Read (读控制):**
 - 用途: 一个1位的控制信号, 当为1时, 启动内存读操作。
 - 来源: 控制逻辑。仅当icode为**mrmovq, popq, ret**时为1。
- **Mem. Write (写控制):**
 - 用途: 一个1位的控制信号, 当为1时, 启动内存写操作。
 - 来源: 控制逻辑。仅当icode为**rmmovq, pushq, call**时为1。
- **Mem. Data Out (读出数据 / valM):**
 - 用途: 从内存中读出的数据。这个值将通过valM信号线传递到写回阶段。

7. Memory Data 和 Memory Write 生成逻辑

这两个信号决定了内存写操作的内容和时机。

Memory Write (内存写控制) 生成逻辑

```
bool MemWrite = icode in { RMMOVQ, PUSHQ, CALL };
```

- 分析:
 - **RMMOVQ rA, D(rB)**: 将寄存器**rA**的值写入内存。需要写内存。
 - **PUSHQ rA**: 将寄存器**rA**的值压入栈，即写入内存。需要写内存。
 - **CALL Dest**: 将返回地址 (**valP**) 压入栈，即写入内存。需要写内存。
 - 其他指令如**MRRMOVQ**, **POPQ**, **RET**是读内存，而**OPQ**, **RMMOVQ**等不访问内存。因此，只有这三条指令需要激活**MemWrite**信号。

Memory Data (写入数据) 生成逻辑

```
word MemDataIn = [
    icode in { RMMOVQ, PUSHQ } : valA;
    icode in { CALL } : valP;
    1 : 0; // 无意义
];
```

- 分析:
 - **icode in { RMMOVQ, PUSHQ } : valA**:
 - **rmmovq rA, D(rB)**: 要写入内存的数据是源寄存器**rA**的值，即**valA**。
 - **pushq rA**: 要压栈的数据是源寄存器**rA**的值，即**valA**。
 - **icode in { CALL } : valP**:
 - **call Dest**: **call**指令需要将紧随其后的指令地址（即返回地址**valP**）保存到栈中。因此，要写入内存的数据是**valP**。
 - 在SEQ设计中，**valP**可以通过**valA**的通路传递到内存阶段，或者设计一个专门的多路选择器来提供内存输入数据。通常为了简化，可以认为**valA**在**call**指令时被设置为**valP**的值。

8. PC Update (PC更新) 部件生成逻辑

PC Update部件负责在每个时钟周期结束时计算下一条指令的地址，并更新PC寄存器。

其生成逻辑如下：

```
word new_pc = [
    // 对于 call 指令，新PC是立即数 valC
    icode == ICALL : valC;
    // 对于条件跳转指令 (jXX)，如果条件满足 (Cnd)，新PC是立即数 valC
    icode == IJXX && Cnd : valC;
    // 对于 ret 指令，新PC是从内存中读出的返回地址 valM
    icode == IRET : valM;
    // 对于其他所有指令，新PC是下一条指令的地址 valP
    1 : valP;
];
```

- **icode == ICALL : valC**: 对于call指令，程序流无条件跳转到指令中指定的地址valC。
- **icode == IJXX && Cnd : valC**: 对于条件跳转指令jXX，首先检查ALU设置的条件码（Cnd信号）。如果条件满足，程序流跳转到指令中指定的地址valC。
- **icode == IRET : valM**: 对于ret指令，程序流返回到调用点。返回地址在内存阶段从栈中读出，并通过valM信号传递过来。
- **1 : valP**: 对于所有其他指令（包括不跳转的jXX指令），处理器顺序执行。新的PC值是取指阶段计算出的紧随当前指令的地址valP。

9. SEQ执行一条指令的完整过程与时序变化

SEQ是一个单周期处理器，所有操作都在一个时钟周期内完成。但其内部包含的**时序逻辑电路**（PC寄存器、条件码寄存器、数据内存、寄存器文件）的状态只在**时钟的上升沿**发生变化。

我们以addq %rax, %rbx (icode:ifun = 6:0, rA=0, rB=3) 为例，讲解其完整过程：

初始状态 (时钟周期开始前):

- **PC**: 指向addq指令的地址，例如0x100。
- **CC** (条件码寄存器): 保存着上一条指令的结果。
- **RegFile** (寄存器文件): %rax存有价值V_A, %rbx存有价值V_B。

时钟周期内 (组合逻辑的传播过程):

1. Fetch (取指):

- PC (0x100) 送入指令存储器。
- 读出指令字节: M_1[0x100]得到60 (icode=6, ifun=0), M_1[0x101]得到03 (rA=0, rB=3)。
- valP被计算为 $0x100 + 2 = 0x102$ 。

2. Decode (译码):

- icode=6，控制逻辑判断出是OPq指令。
- srcA设为rA (0, %rax)，srcB设为rB (3, %rbx)。
- dstE设为rB (3, %rbx)，dstM设为RNONE。
- 寄存器文件根据srcA和srcB立即读出值: valA = V_A, valB = V_B。

3. Execute (执行):

- aluA设为valA (V_A)。
- aluB设为valB (V_B)。
- ALUFunc根据ifun=0设为加法操作。
- ALU计算 $valE = valA + valB = V_A + V_B$ 。
- 同时，ALU根据计算结果更新条件码，生成新的CC值（例如，如果结果为0，ZF位置1）。

4. Memory (访存):

- icode=6，控制逻辑将Mem.Read和Mem.Write都设为0。
- 内存地址和写入数据信号被忽略，内存无操作。
- valM无有效值。

5. Write Back (写回):

- `dstE`为3 (`%rbx`), `valE`为 $V_A + V_B$ 。`valE`被送到寄存器文件的`valE`输入端。
- `dstM`为`RNONE`, `valM`的写回被忽略。

6. PC Update (PC更新):

- `icode=6`, 不满足`call`, `jxx`, `ret`的条件。
- 新的PC值`new_pc`被选择为`valP`, 即`0x102`。

时钟上升沿 (状态更新):

在下一个时钟周期的上升沿, 所有时序逻辑部件同时更新:

- **PC寄存器:** `PC`的值从`0x100`更新为`new_pc`的值`0x102`。
- **条件码寄存器 (CC):** `CC`的值更新为ALU在执行阶段计算出的新条件码。
- **寄存器文件 (RegFile):** `%rbx`的值被更新为`valE`的值 $V_A + V_B$ 。
- **数据内存:** 无变化, 因为没有写操作。

至此, 一条`addq`指令的执行周期完成。处理器已经准备好在下一个时钟周期从`0x102`地址取下一条指令。